# Vídeo 5.1: Nem todo dado de fora é confiável

**Duração-alvo:** 5 min | **Conceito:** por que validar | **O aluno sai sabendo:** enxergar os problemas que dados ruins causam e por que a API precisa conferi-los.

**Antes de gravar:** partir do código do fim do módulo 4 (versão v4) com o servidor rodando e preparar os três pedidos ruins no requisicoes.http. Conferir a versão do Express instalada: este roteiro assume o Express 5, em que um erro dentro de uma rota async chega ao tratador de erros do Express e vira uma resposta 500. Em versões anteriores, o pedido pode ficar sem resposta.

**[0:00] Abertura**

*Na tela:* título do vídeo.

*Fala:* “A nossa API já guarda os livros de verdade. Mas ela tem um ponto fraco: confia em tudo que chega. Neste vídeo vamos descobrir o que acontece quando o cliente manda dados errados. A boa notícia é que, depois de ver o problema, a solução fica fácil de entender.”

**[0:40] A ficha em branco**

*Na tela:* ilustração de uma ficha de cadastro em branco sendo entregue ao balcão.

*Fala:* “Imagine que alguém entrega no balcão uma ficha de cadastro em branco. Uma atendente atenta devolve e diz: preencha o título. Uma atendente distraída guarda a ficha em branco no fichário. Em pouco tempo, o fichário vira uma bagunça. Quem confere é você, o servidor. Regra de ouro do back-end: tudo que vem de fora, do cliente, pode vir errado, por engano ou de propósito. Nunca confie sem conferir.”

**[1:40] Três pedidos que atrapalham a API**

*Na tela:* no requisicoes.http, escrever os três pedidos abaixo e enviar um de cada vez. Depois, listar os livros.

```
### Sem o autor
POST http://localhost:3000/livros
Content-Type: application/json

{ "titulo": "Livro sem autor" }

### Título vazio
POST http://localhost:3000/livros
Content-Type: application/json

{ "titulo": "", "autor": "Alguém" }

### Título que não é texto
POST http://localhost:3000/livros
Content-Type: application/json
```

{ "titulo": 12345, "autor": "Alguém" }
*Fala:* “Vou fazer três pedidos ruins. Primeiro, um livro sem autor. A resposta é 500, erro interno. Veja: o servidor disse que o erro foi dele, mas o erro foi do pedido. O código certo seria um 400, pedido inválido. Segundo, um título vazio: o servidor aceitou e cadastrou um livro sem nome. Terceiro, um título que é um número: o servidor também aceitou, sem reclamar. Listo os livros, e lá estão o livro sem nome e o livro com título 12345, sujando o nosso banco.”

**[3:20] O que a API deveria fazer**

*Na tela:* três itens aparecendo um de cada vez.

*Fala:* “O que queremos? Primeiro, conferir os dados antes de guardar. Segundo, quando algo estiver errado, responder com 400 e uma mensagem clara dizendo o que corrigir. Terceiro, guardar só o que está certo. Conferir os dados tem um nome técnico: validação.”

**[4:10] Como organizar a conferência**

*Na tela:* a palavra “middleware” aparecendo.

*Fala:* “Vamos escrever a conferência uma vez e usá-la em todas as rotas que precisam. Para isso existe uma ferramenta do Express chamada middleware. É o assunto do próximo vídeo.”

**[4:30] Resumo**

*Na tela:* uma frase: tudo que vem de fora deve ser conferido.

*Fala:* “Resumindo: dados que vêm de fora podem estar errados. Sem conferir, a API devolve erros que não ajudam e guarda lixo no banco. A conferência se chama validação. Até o próximo vídeo!”

**Exercício sugerido:** listar três dados ruins que alguém poderia mandar para o cadastro de um livro e dizer qual resposta a API deveria dar para cada um.